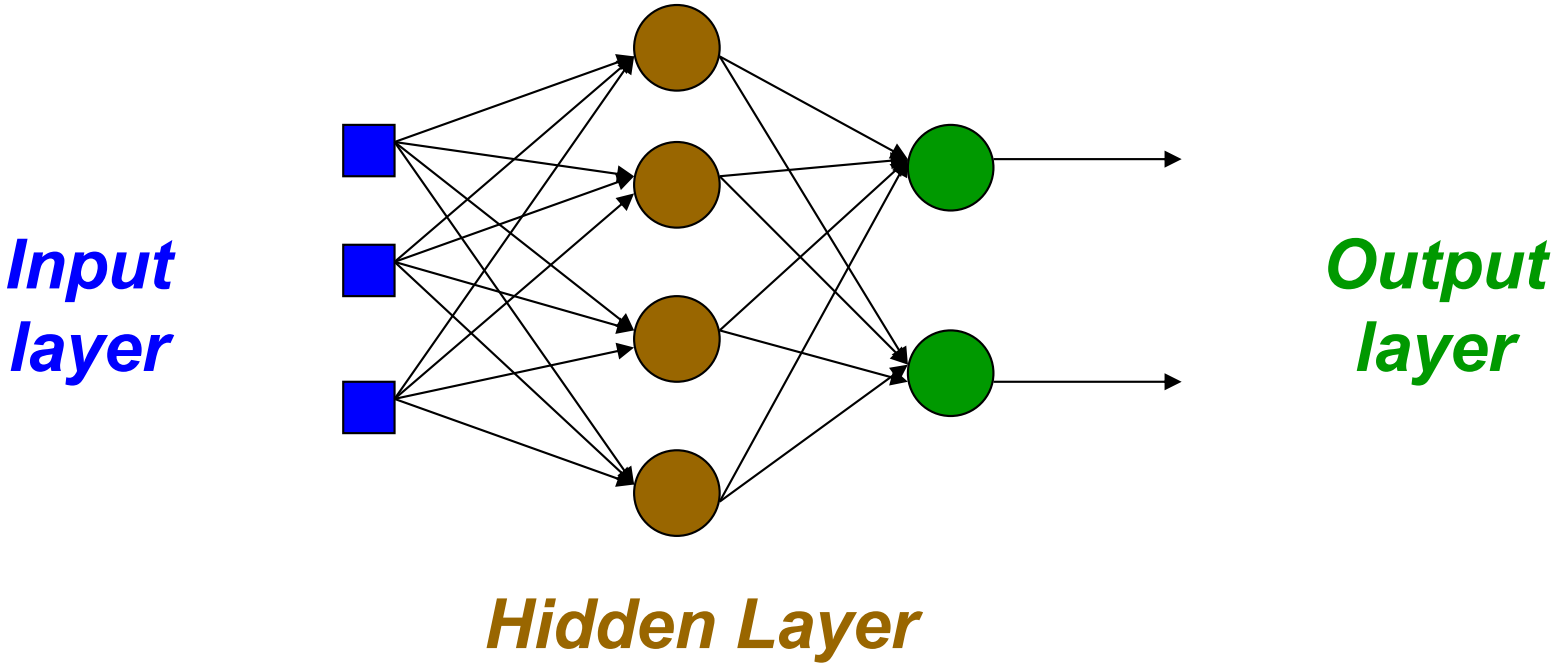
Feed Forward Neural Networks

Outline

- Multi-layer Neural Networks
- Feedforward Neural Networks
  - FF NN model
  - Backpropogation (BP) Algorithm
  - Practical Issues of FFNN

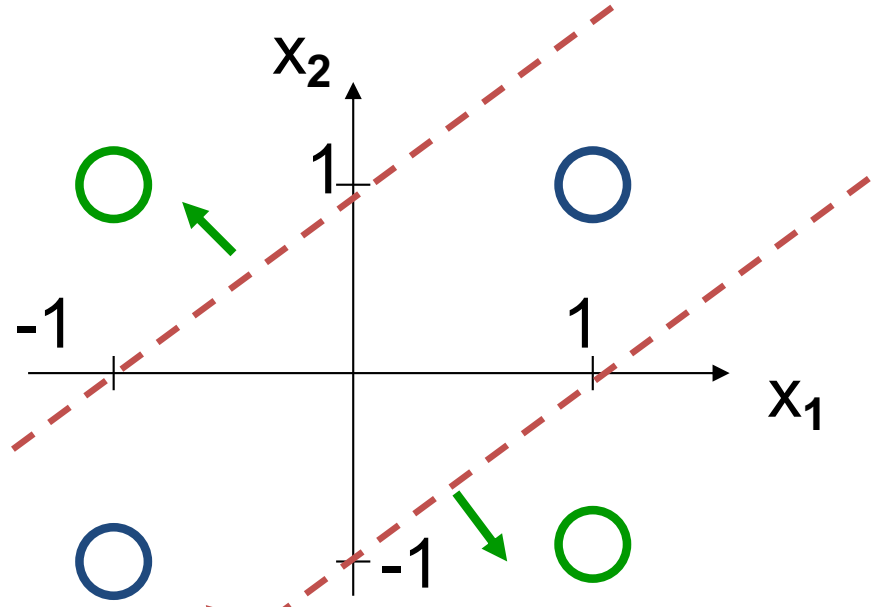
Multi-layer NN

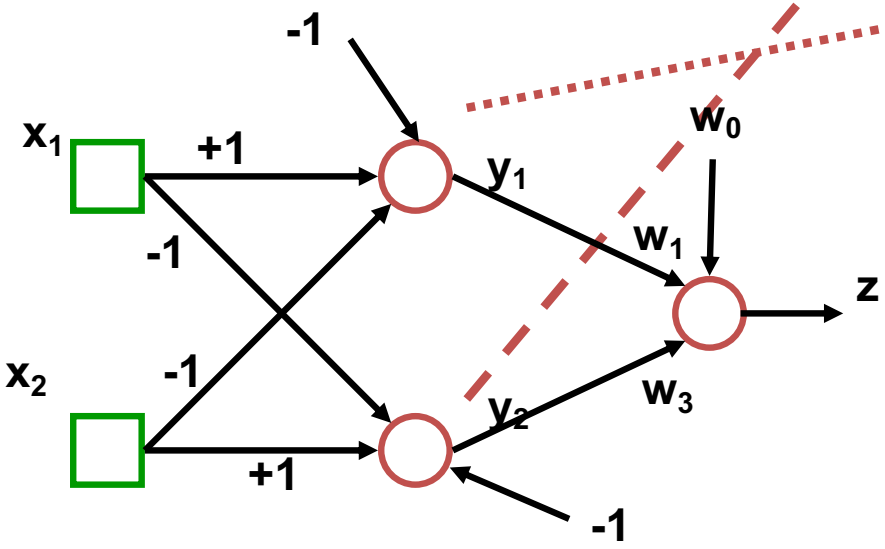
- Between the input and output layers there are hidden layers, as illustrated below.
  - Hidden nodes do not directly send outputs to the external environment.
- Multi-layer NN overcome the limitation of a single-layer NN
  - they can handle non-linearly separable learning tasks.



XOR problem

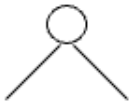
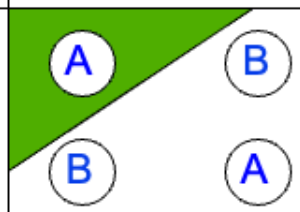
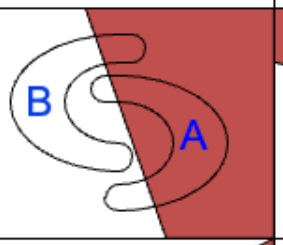
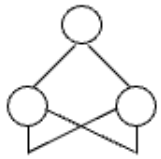
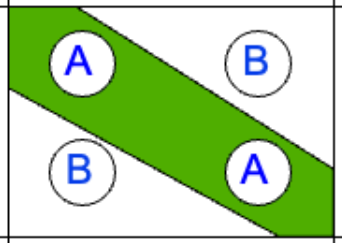
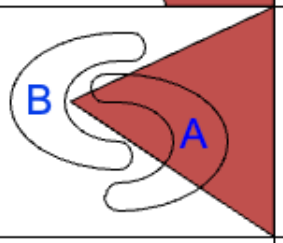
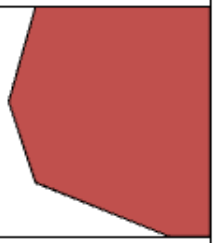
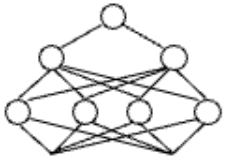
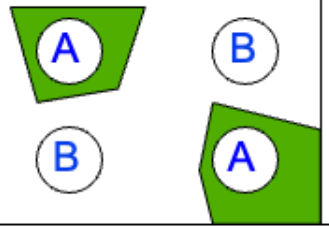
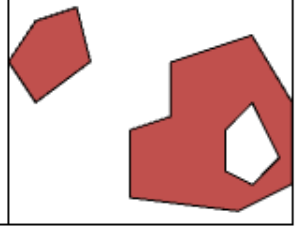
Two classes, green and blue, cannot be separated using one line, but **two lines**. The NN below with **two hidden nodes** realizes this non-linear separation, where each hidden node represents one of the two red lines.





Types of decision regions

- NN = learn complex decision boundaries automatically
  - Tree → axis-aligned
  - SVM → margin-based
  - NN → arbitrary smooth boundary

Structure	*Types of Decision Regions*	*Exclusive-OR Problem*	*Class Separation*	*Most General Region Shapes*
Single-Layer 	*Half Plane Bounded By Hyperplane*			
Two-Layer 	*Convex Open Or Closed Regions*			
Three-Layer 	Arbitrary (Complexity Limited by No. of Nodes)			

Outline

- Multi-layer Neural Networks
- Feedforward Neural Networks (FFNN) Model
  - FF NN model
  - Backpropogation (BP) Algorithm
  - Practical Issues of FFNN

Neuron Model (Feedforward Neural Network)

- A neuron computes a **weighted sum + activation**:

$$z = w^T x + b$$
$$a = f(z)$$

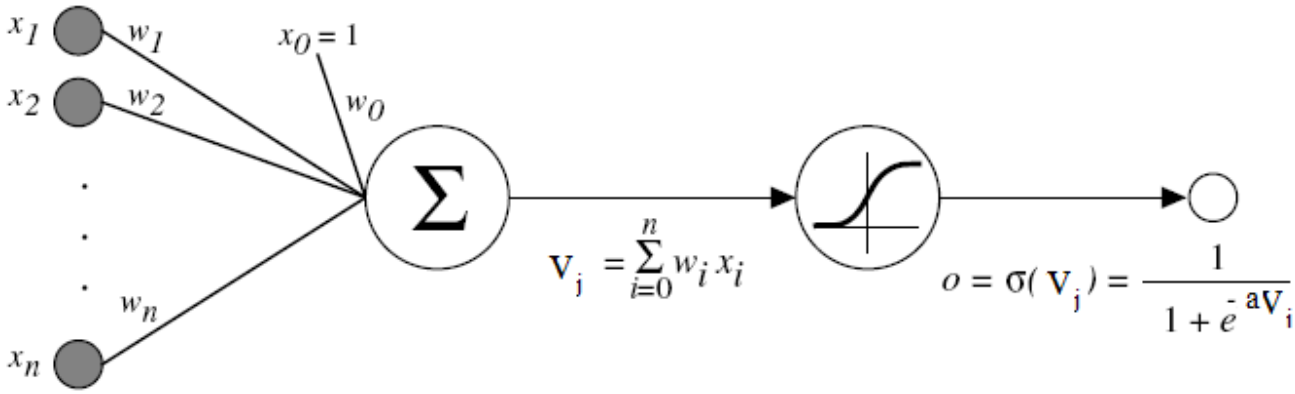
- Components:

Input: x

Weights: w

Bias: b

Activation function: $f(\cdot)$



Activation Functions: Why Do We Need Them?

- Without activation → model is **linear**
- With activation → model becomes **non-linear**
- Enables learning **complex decision boundaries**

- *Choice of activation function strongly affects training performance*

Sigmoid Activation Function

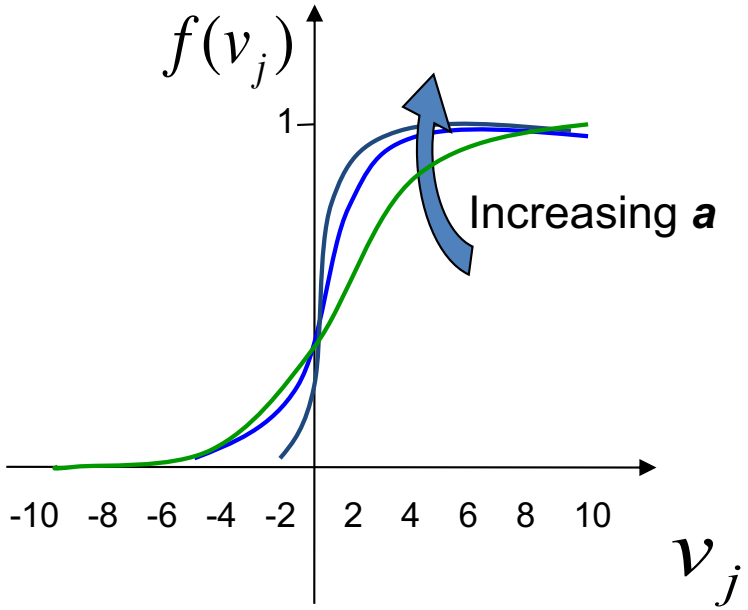
- Formula:

$$f(\mathrm{v_j}) = \frac{1}{1+e^{-av_j}} \quad \text{with } a > 0$$

where $\mathrm{v_j} = \sum_i w_{ji} y_i$

with w_{ji} weight of link from node i

to node j and y_i output of node i

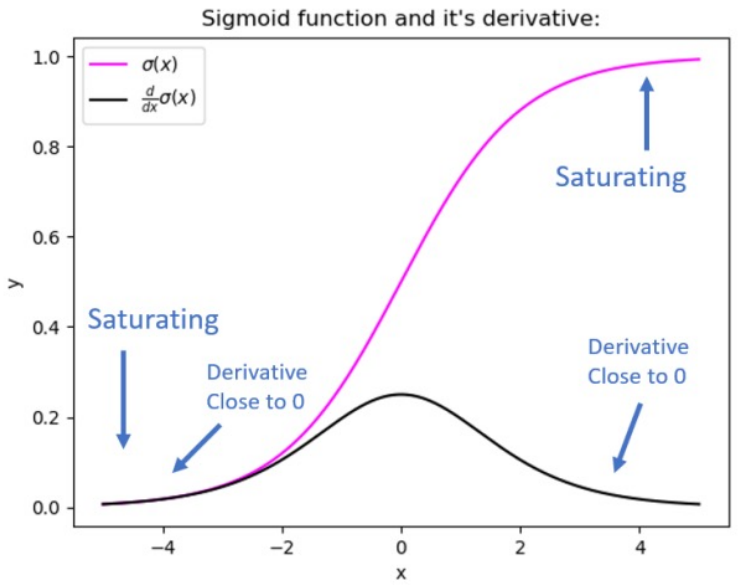


  - when a=1, $f(x) = \frac{1}{1+e^{-x}}$
  - when **a** approaches to 0, f tends to a linear function
  - when **a** tends to infinity then f tends to the step function

- Output range: (0, 1)
- Interpreted as probability
- Used mainly in: *Binary classification output layer*

Limitations of Sigmoid (and tanh)

- Saturation:
  - Large |x| → gradient ≈ 0

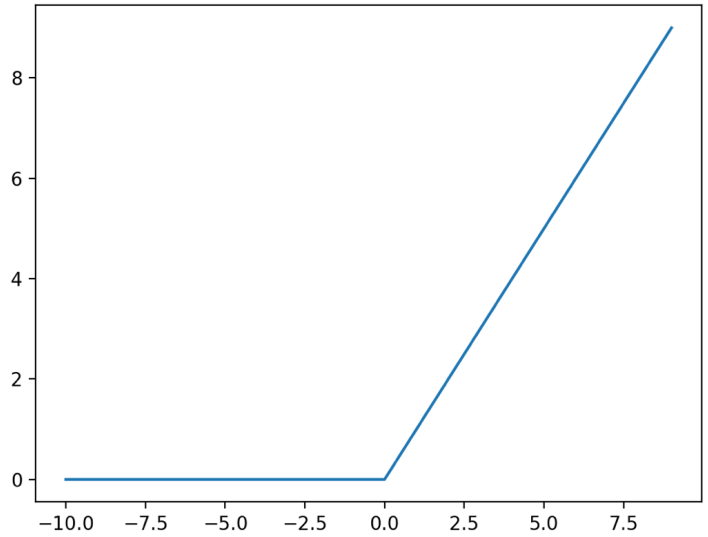


- Vanishing gradient:
  - Slows or stops learning in deep networks
- Output not zero-centered

- *Sigmoid is rarely used in hidden layers today*

ReLU: The Default Activation Function

- ReLU:
  - *f(x) = max(0, x)*
  - Derivative:
    - 0 for x <= 0;
    - 1 for x > 0
- Properties:
  - Non-linear function
  - Efficient to compute
  - Avoids vanishing gradient (for positive region)
  - Works well for **deep networks**
  - Enables faster training
- Limitation: Dying ReLU problem (neurons stuck at 0)



piece-wise linear

Other Activation Functions

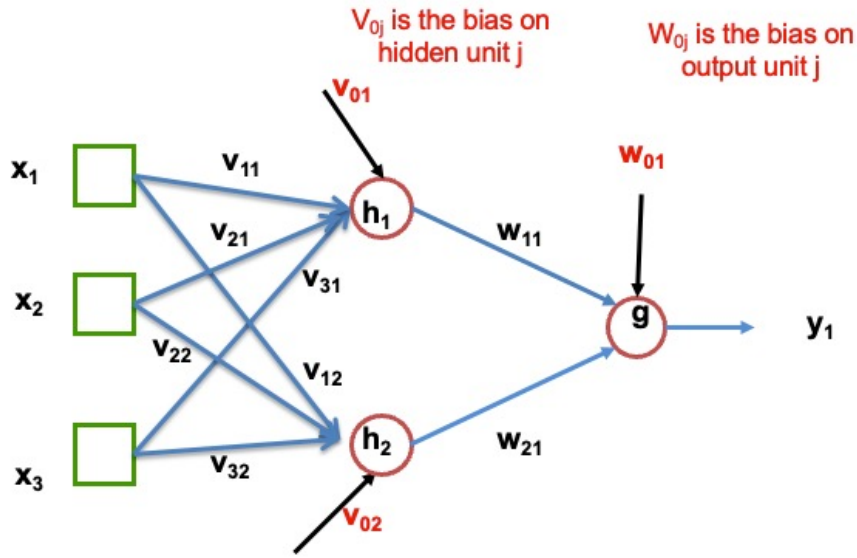
- Leaky ReLU:
  - Fixes dying ReLU
- Softmax:
  - Used for multi-class classification
  - Converts outputs → probabilities

Function	Use
ReLU	Hidden layers
Sigmoid	Binary output
Softmax	Multi-class output

Feedforward Neural Network Model

- A feedforward neural network computes outputs layer by layer:
  - **Input layer** receives features
  - **Hidden layer(s)** transform the representation
  - **Output layer** produces prediction

x_i : inputs
h_j : hidden activations
y^k : outputs
v_{ij} : input→hidden weights
w_{jk} : hidden→output weights



Feedforward Neural Network Model

Hidden unit input: $z_j = \sum_i v_{ij} x_i + v_{0j}$

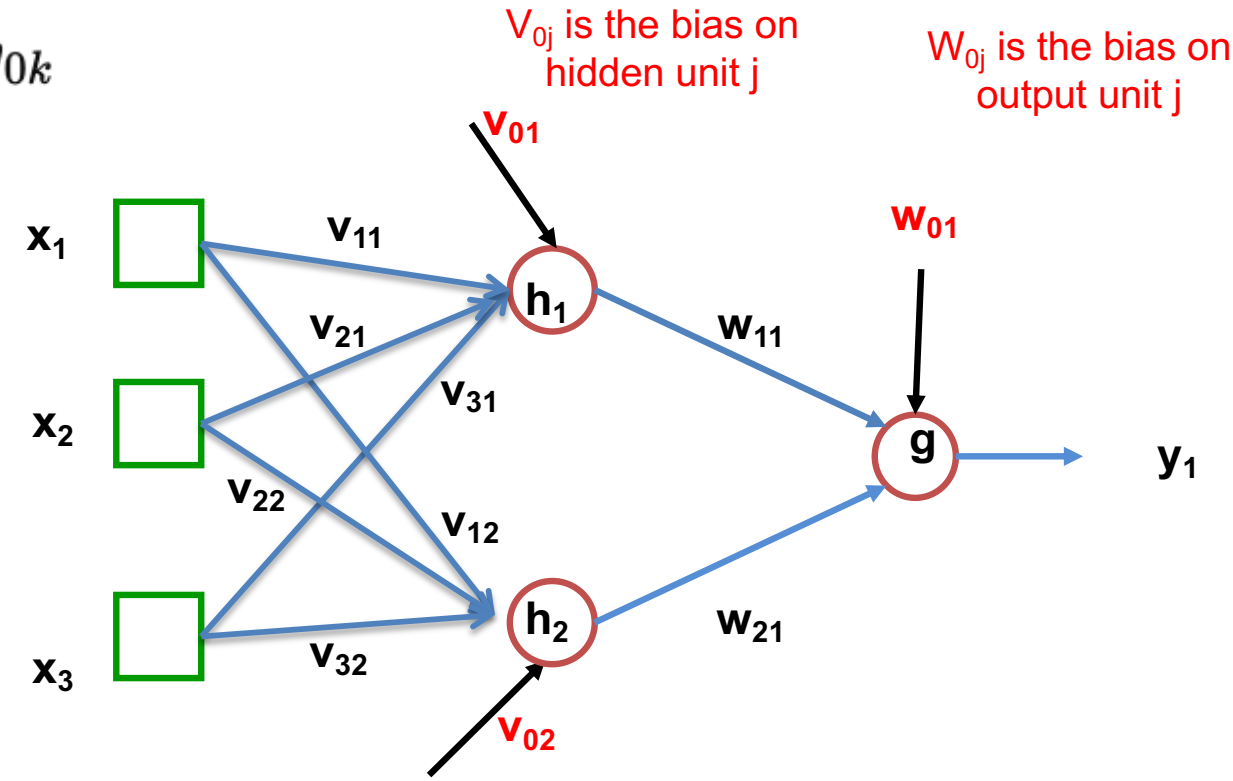
Hidden unit output: $h_j = f(z_j)$

Output unit input:

$$s_k = \sum_j w_{jk} h_j + w_{0k}$$

Output:

$$\hat{y}_k = g(s_k)$$



How Do We Measure Error?

- After the network produces prediction $\hat{y}$, we compare it with the target y
- The loss function measures how wrong the prediction is
- For **regression**: $$L = \frac{1}{2}\sum_{k}(y_k - \hat{y}_k)^2$$
- For a dataset of N examples: $$J(W) = \frac{1}{N}\sum_{n=1}^{N} L^{(n)}$$
- Training aims to find weights that **minimize the total loss**

How Do We Measure Error?

- For classification, **cross-entropy loss** is usually preferred over MSE
- Binary Classification (sigmoid output)

$$L = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$$

  - y: true label (0 or 1)
  - $\hat{y}$: predicted probability
- Multi-Class Classification (softmax output)

$$L = -\sum_{k} y_k \log(\hat{y}_k)$$

  - y_k=*1* for correct class, 0 otherwise

Softmax for Multi-Class Classification

- For multi-class classification, we use K output neurons, where K = number of classes
- Before Softmax:
  - h_j: hidden layer outputs
  - z_k: **logit (raw score)** for class k, output layer
  - w_{jk}: weights

$$z_k = \sum_j w_{jk} h_j + b_k$$

- Apply Softmax:
  - Converts outputs into **probabilities**
  - Outputs sum to **1**
  - Each value in **[0,1]**

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Softmax picks the most likely class by comparing all outputs

The Credit Assignment Problem

- When the network makes an error, how do we determine:
  - which hidden units contributed to the error?
  - which weights should change?
  - by how much?

  This is called the **credit assignment problem**

  Backpropagation solves this by propagating the error backward through the network
- Idea: assign responsibility for the final error to internal weights using the **chain rule**

Outline

- Multi-layer Neural Networks
- Feedforward Neural Networks
  - FF NN model
  - Backpropogation (BP) Algorithm
  - Practical Issues of FFNN

Training a Neural Network

- Forward pass
- Loss computation
- Backward pass
- Weight update

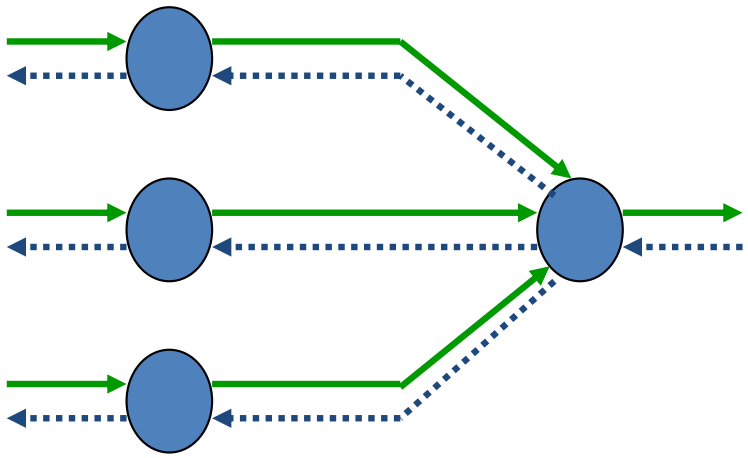
Backpropagation: Main Idea

- Backpropagation is used to train a neural network by minimizing the loss
- Training repeats four steps:
  1. Forward pass: compute activations and outputs
  2. Loss computation: compare prediction with target
  3. Backward pass: compute gradients using chain rule
  4. Weight update: adjust weights to reduce loss

$$w \leftarrow w - \eta \frac{\partial J}{\partial w} \qquad \eta = \text{learning rate}$$

Training Loop: Forward Pass and Backward Pass

- Forward and backward passes illustrated:



Network activation
Error computation
Forward Step

Error propagation
Backward Step

Training Loop: Forward Pass and Backward Pass

- **Forward pass**:
  - Inputs move from left to right
  - Compute hidden activations
  - Compute output prediction
  - Compute loss
- **Backward pass**: The network error is used for updating the weights (**credit assignment problem**).
  - Start from the output error
  - Propagate gradients backward
  - Compute gradient for each weight
  - Update weights

Backpropagation at the Output Layer

For an output unit k:

$$\delta_k = (\hat{y}_k - y_k)\, g'(s_k)$$

Weight update from hidden unit j to output unit k:

$$\Delta w_{jk} = -\eta\, \delta_k\, h_j$$

Bias update:

$$\Delta w_{0k} = -\eta\, \delta_k$$

δ_k is the output unit's **error signal**

It tells us how strongly this output unit contributed to the loss

Backpropagation at the Hidden Layer

- A hidden unit does not directly see the target value, so its error must be computed indirectly.

For hidden unit j: $\delta_j = f'(z_j) \sum_k w_{jk} \delta_k$

Weight update from input i to hidden unit j:

$$\Delta v_{ij} = -\eta \, \delta_j \, x_i$$

Bias update:

$$\Delta v_{0j} = -\eta \, \delta_j$$

- Hidden-layer error is computed from the errors of the next layer, This is why the method is called **backpropagation**

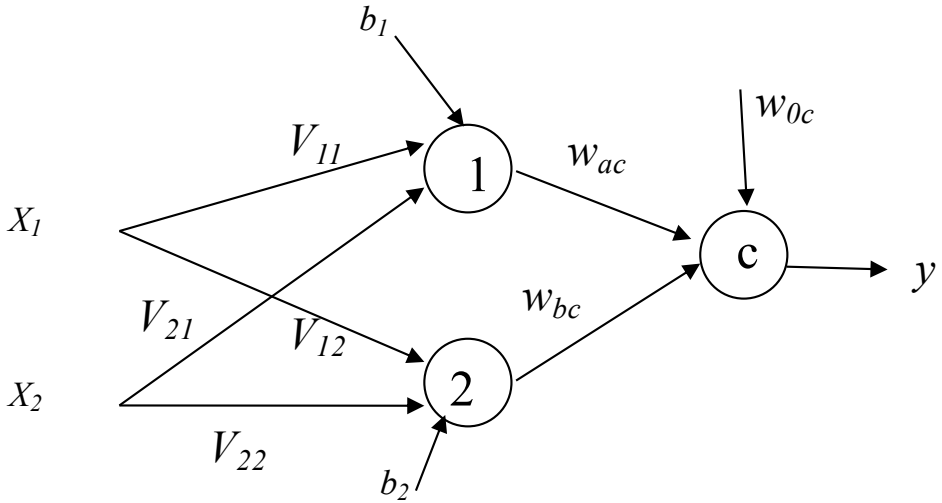
Example: Learning XOR with a Neural Network

- XOR is **not linearly separable**

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

For this example, we use **sigmoid activation**. In practice, we typically use ReLU in hidden layers, but sigmoid is easier for illustrating backpropagation.

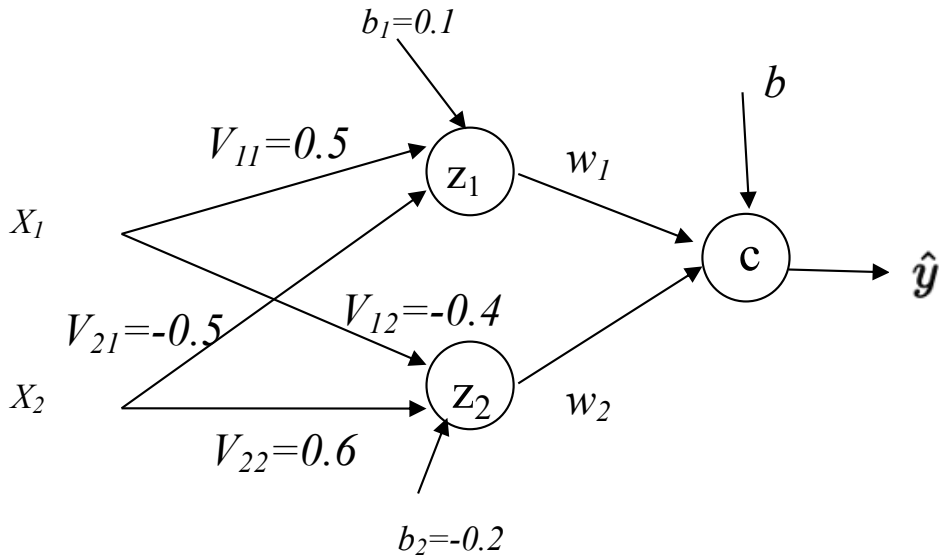
- Requires at least **one hidden layer**



XOR Example: Forward Pass

Use: $x = (1,0)$, target $y = 1$
Assume weights:

- Input → hidden:
  - $v_{11} = 0.5,\ v_{21} = -0.5$
  - $v_{12} = -0.4,\ v_{22} = 0.6$
- Biases:
  - $b_1 = 0.1,\ b_2 = -0.2$



Compute hidden layer

$$z_1 = 0.5(1) + (-0.5)(0) + 0.1 = 0.6$$

$$h_1 = \sigma(0.6) \approx 0.65$$

$$z_2 = -0.4(1) + 0.6(0) - 0.2 = -0.6$$

$$h_2 = \sigma(-0.6) \approx 0.35$$

XOR Example: Output and Loss

Output weights:

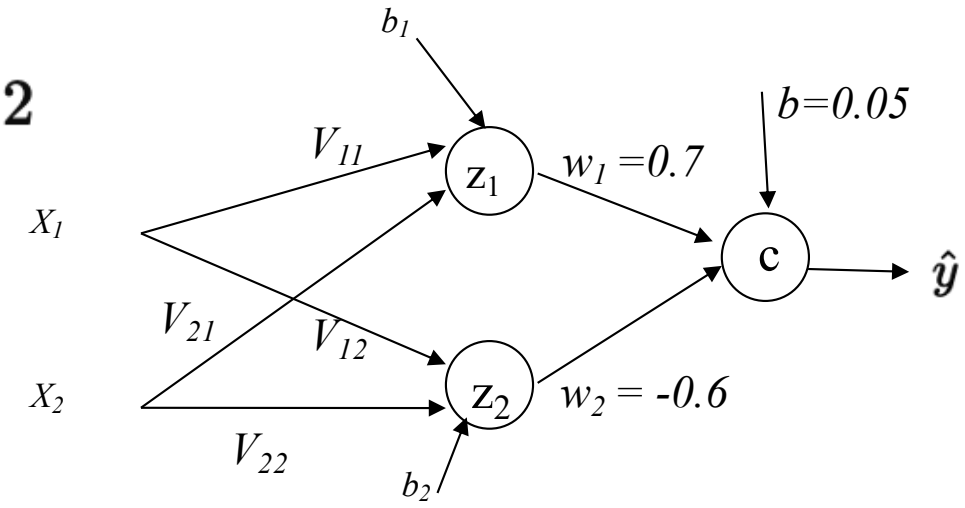
- $w_1 = 0.7,\ w_2 = -0.6$
- bias $b = 0.05$

$$s = 0.7(0.65) + (-0.6)(0.35) + 0.05 \approx 0.455 - 0.21 + 0.05 = 0.295$$

$$\hat{y} = \sigma(0.295) \approx 0.57$$

Loss

$$L = \frac{1}{2}(1 - 0.57)^2 \approx 0.092$$



XOR Example: Backprop (Output Layer)

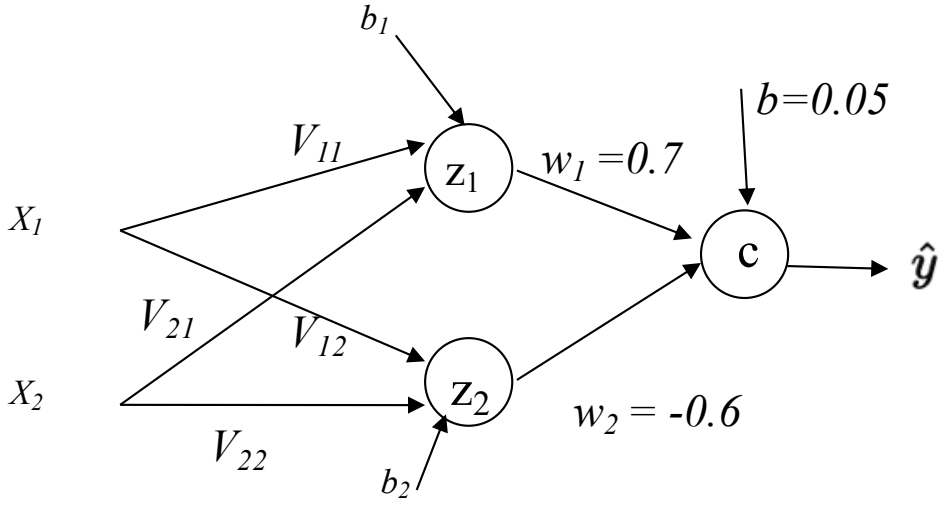
$$\delta = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y})$$

$$\delta = (0.57 - 1) \cdot 0.57(0.43) \approx (-0.43)(0.245) \approx -0.105$$

Weight updates

$$\Delta w_1 = -\eta \cdot \delta \cdot h_1$$

$$\Delta w_2 = -\eta \cdot \delta \cdot h_2$$



Assume $\eta = 0.5$:

$$\Delta w_1 \approx -0.5(-0.105)(0.65) \approx +0.034$$

$$\Delta w_2 \approx -0.5(-0.105)(0.35) \approx +0.018$$

XOR Example: Backprop (Hidden Layer)

$$\delta_j = h_j(1 - h_j) \cdot w_j \cdot \delta$$

For h_1:

$$\delta_1 = 0.65(0.35)(0.7)(-0.105) \approx -0.017$$

For h_2:

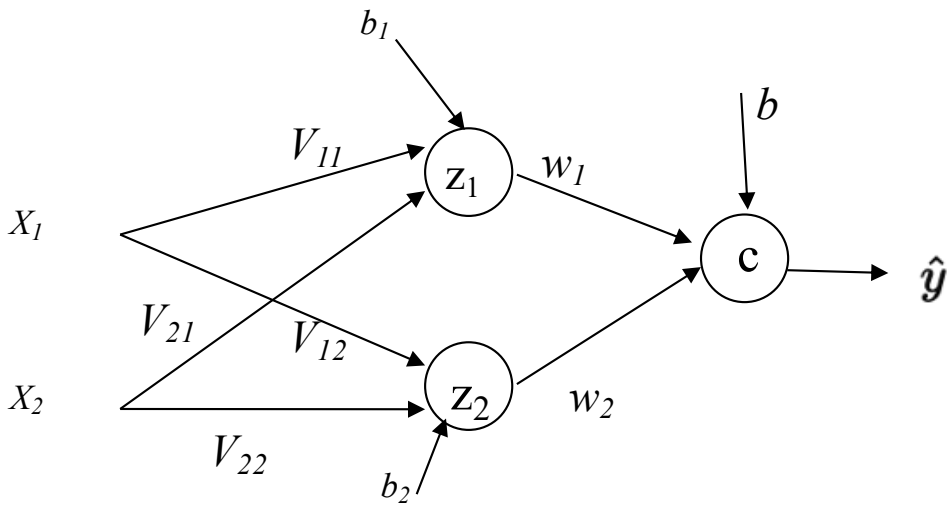
$$\delta_2 = 0.35(0.65)(-0.6)(-0.105) \approx 0.014$$

Update input weights

$$\Delta v_{11} = -\eta \cdot \delta_1 \cdot x_1 \approx -0.5(-0.017)(1) = +0.0085$$

XOR Example: One Training Iteration

- Steps completed:
  1. Forward pass → compute prediction
  2. Compute loss
  3. Backprop:
     - Output layer error
     - Hidden layer error
  4. Update weights



Forward & Backward Pass with ReLU

- Input: $x = (2, -1)$
- Weights (input → hidden): $w_1 = 1, \quad w_2 = 1$
- Bias: $b = -1$
- Forward Pass
  - Forward Pass $z = 1 \cdot 2 + 1 \cdot (-1) - 1 = 0$
  - ReLU activation $h = \max(0, z) = 0$
- Output Layer
  - Weight: $w = 2, \quad b = 0$

$$\hat{y} = 2 \cdot 0 = 0$$

Forward & Backward Pass with ReLU

- Assume target: $y = 1$
- Loss (MSE) $L = \frac{1}{2}(1-0)^2 = 0.5$

- Backward Pass
  - Output gradient $\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = -1$
  - ReLU derivative

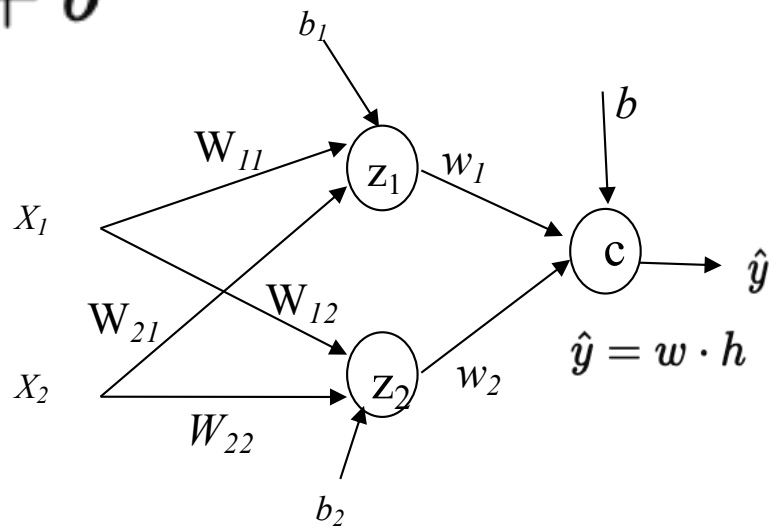
$$\frac{d}{dz}\text{ReLU}(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

  - z=0 → derivative = **0**
  - **Let h=ReLu(z)** $z = 0 \Rightarrow \frac{\partial h}{\partial z} = 0$

Forward & Backward Pass with ReLU

- Linear part $z = w_1 x_1 + w_2 x_2 + b$

$$\frac{\partial z}{\partial w_1} = x_1$$



- Apply Chain rule:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h} \cdot \frac{\partial h}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$

$$= (\hat{y} - y) \cdot w \cdot \underbrace{0}_{\text{ReLU derivative}} \cdot x_1 = 0$$

$$\frac{\partial L}{\partial w_1} = 0 \quad \frac{\partial L}{\partial w_2} = 0$$

- "Dying ReLU" is when a neuron outputs zero for all inputs ($z \leq 0$), causing zero gradients and preventing it from ever updating again.

Common Solutions to Dying ReLU

- Use **Leaky ReLU** $f(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases} \quad (\alpha \approx 0.01)$
  - Small slope for negative values
  - Gradient is **never zero**
- Proper initialization (He initialization)

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$$

  - Helps ensure:
    - enough neurons start with $z > 0$
    - reduces chance of early death
- Use a reasonable learning rate
- Others

Training Modes

Method	Description
Batch	update after full dataset
Stochastic (SGD)	update per example
Mini-batch	update per small batch

Mini-batch is the **standard in practice**

Choosing Network Size

- Too small → underfitting
- Too large → overfitting
- Increasing hidden units increases model capacity

Outline

- Multi-layer Neural Networks
- Feedforward Neural Networks
  - FF NN model
  - Backpropogation (BP) Algorithm
  - Practical Issues of FFNN

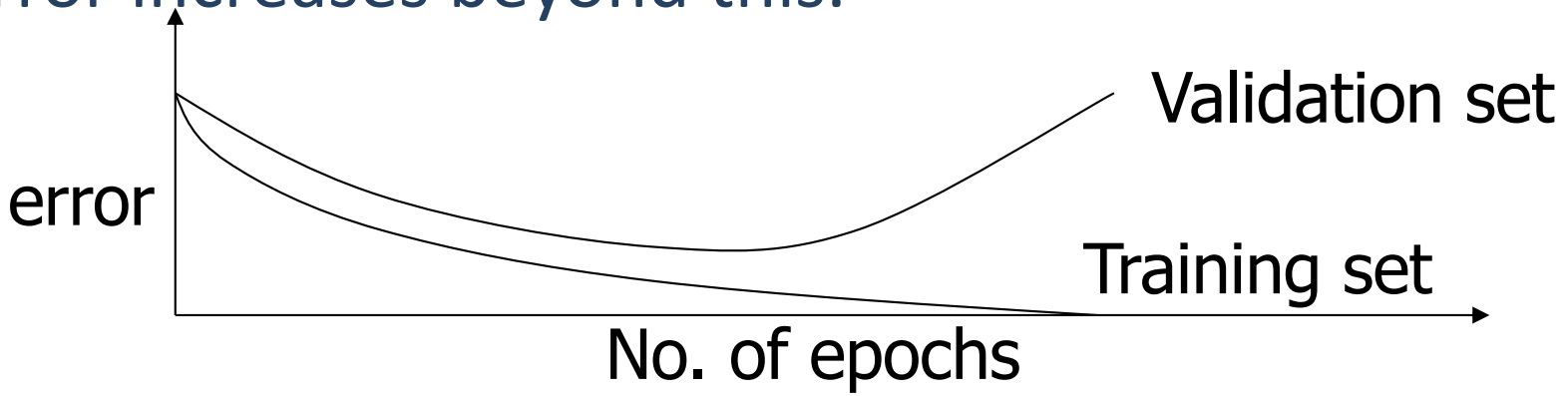
Training, Validation, Test Sets

- The available data set is normally split into three sets as follows:
  - Training set – use to update the weights. Patterns in this set are repeatedly in random order. The weight update equation are applied after a certain number of patterns.
  - Validation set – use to tune model and decide when to stop training by monitoring the error.
  - Test set – use to test the final performance of the neural network. It should not be used as part of the neural network development cycle.

Early Stopping for Generalization

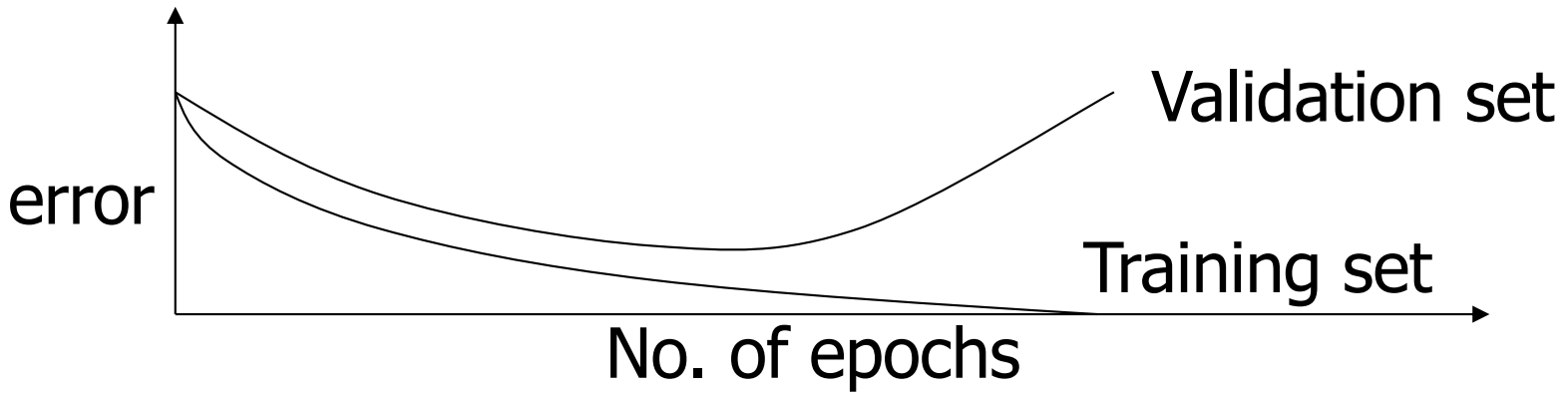
- Training error decreases continuously
- Running too many epochs may overtrain the network and result in overfitting and perform poorly in generalization.

➢ Keep a hold-out validation set and test accuracy after every epoch. Maintain weights for best performing network on the validation set and stop training when error increases beyond this.



Choosing Network Size

- Too few hidden units -- underfitting
  - prevent the network from learning adequately fitting the data and learning the concept.
  - Increasing hidden units increases model capacity
- Too many hidden units – overfitting
  - cross-validation can be used to determine an appropriate number of hidden units



NN DESIGN

- Data representation
- Network Architecture
- Key Hyperparameters
- Training

Data Preprocessing for Neural Networks

- Neural networks work best with **numerical, scaled inputs**
- Common preprocessing steps:
  - Convert categorical features → numeric (one-hot encoding)
  - Handle missing values
  - Normalize or standardize features

$$x_i' = \frac{x_i - \mu_x}{\sigma_x}$$

  - Min-max scaling to [0,1]

$$x_i' = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$$

  - Poor scaling → slow training or unstable learning

Designing Network Architecture

- Key design choices:
  - Number of layers
  - Number of neurons per layer
- General guidelines:
  - Start simple → increase complexity if needed
  - More layers → more expressive power
  - More neurons → higher capacity
- Most tabular problems:
  - 1–3 hidden layers is usually sufficient

Key Hyperparameters in Neural Networks

- Learning rate (η)
- Number of hidden layers
- Number of neurons
- Batch size
- Number of epochs

*Learning rate is often the **most important parameter***

Weight Initialization

- Weight Initialization:
  - Stable training
  - Faster convergence
  - the quality of the local minimum point reached
- Keep variance of activations stable across layers
  - Xavier (Glorot) initialization
    - Good for sigmoid/tanh
    - Keeps the signal stable as it flows through the network.
  - He initialization
    - Designed for ReLU
    - Boosts the signal to compensate for ReLU shutting off neurons, i.e., dropping ~50% of signals.

Learning Rate and Its Impact

- Learning rate controls step size during training:

$$w \leftarrow w - \eta \nabla J$$

  - Too large → training diverges
  - Too small → very slow learning

- Learning rate scheduling:
  - decrease over time
  - improves convergence

Adam adapts learning rate automatically per parameter (weight)

Data Size and Generalization

- More data → better generalization
- Neural networks are **data-hungry models**
- If data is limited:
  - risk of overfitting
  - use regularization to make the model generalize better
    - Add penalty to loss:

$$J = \text{Loss} + \lambda \sum w^2$$

    - Effect:
      - discourages large weights, simpler model, smoother decision boundary

*When data is small, use **regularization to control model complexity***

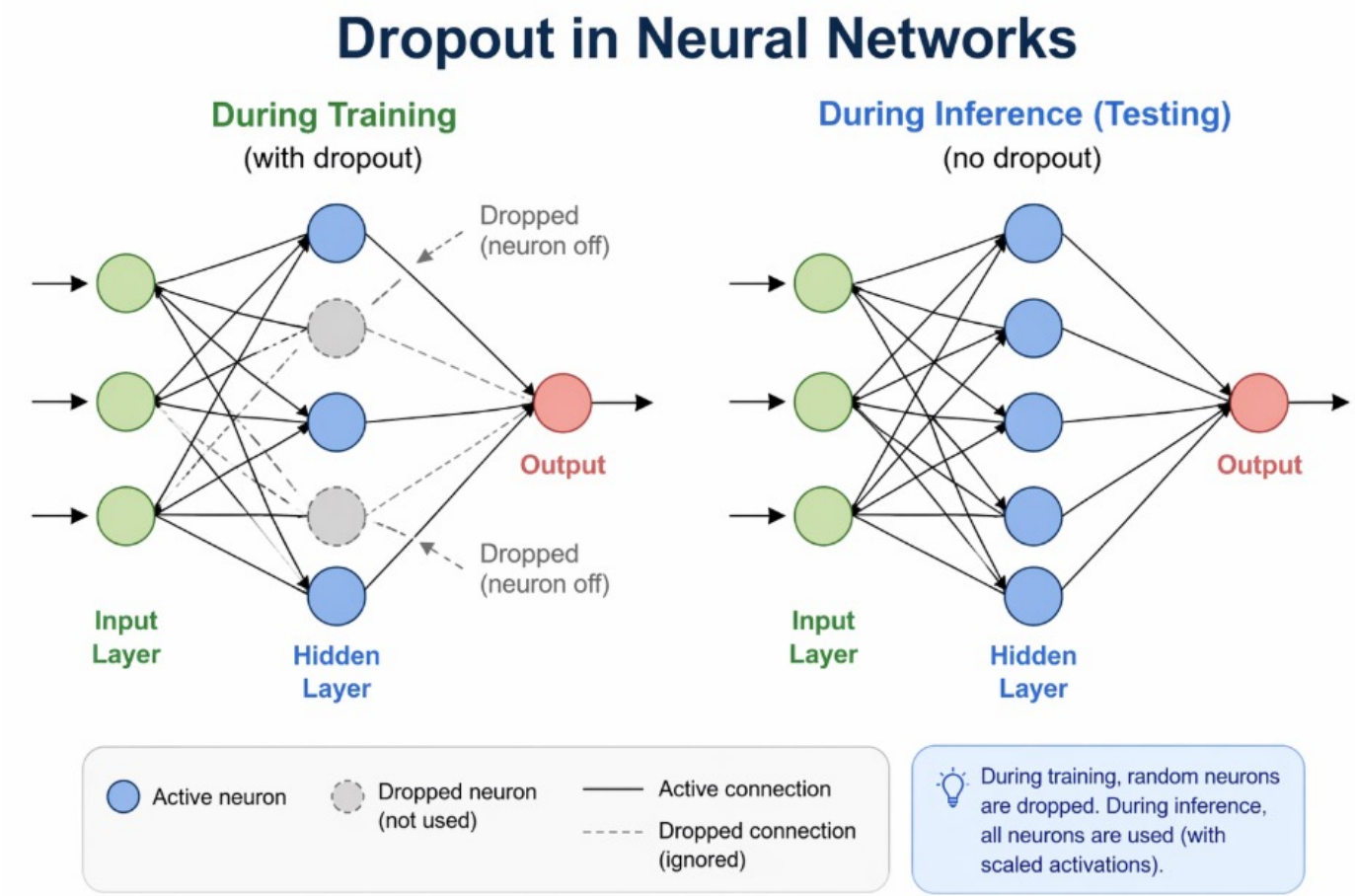
Dropout: A Simple Regularization Technique

- What is Dropout?
  - During training, randomly "turn off" some neurons
  - Each neuron is dropped with probability p
- Why use Dropout?
  - Prevents overfitting
  - Forces network to learn **robust, distributed features**
  - Reduces reliance on specific neurons
- Training → neurons randomly dropped
- Testing → full network used (with scaled activations)

*Each training step uses a **slightly different network***
Similar to averaging many models (ensemble effect)

Dropout: A Simple Regularization Technique

- Training → neurons randomly dropped
- Testing → full network used (with scaled activations)



Applications of Neural Networks

- Classification
  - Image recognition
  - Fraud detection
  - Medical diagnosis
- Regression
  - Price prediction
  - Forecasting
- Modern applications
  - Natural Language Processing (ChatGPT, translation)
  - Recommendation systems
  - Speech recognition

Neural networks are powerful because they can learn ***complex non-linear relationships***

How to Train a Neural Network

Workflow:

1. Preprocess data (scale, encode)
2. Split into train/validation/test
3. Choose architecture
4. Initialize weights
5. Train using mini-batch + optimizer (Adam)
6. Monitor validation loss
7. Tune hyperparameters
8. Evaluate on test set

Neural networks require both good modeling and careful tuning